

Air101入门手册

开发环境搭建

开发环境搭建

烧录与查看日志

烧录教程

GPIO输出（点灯）

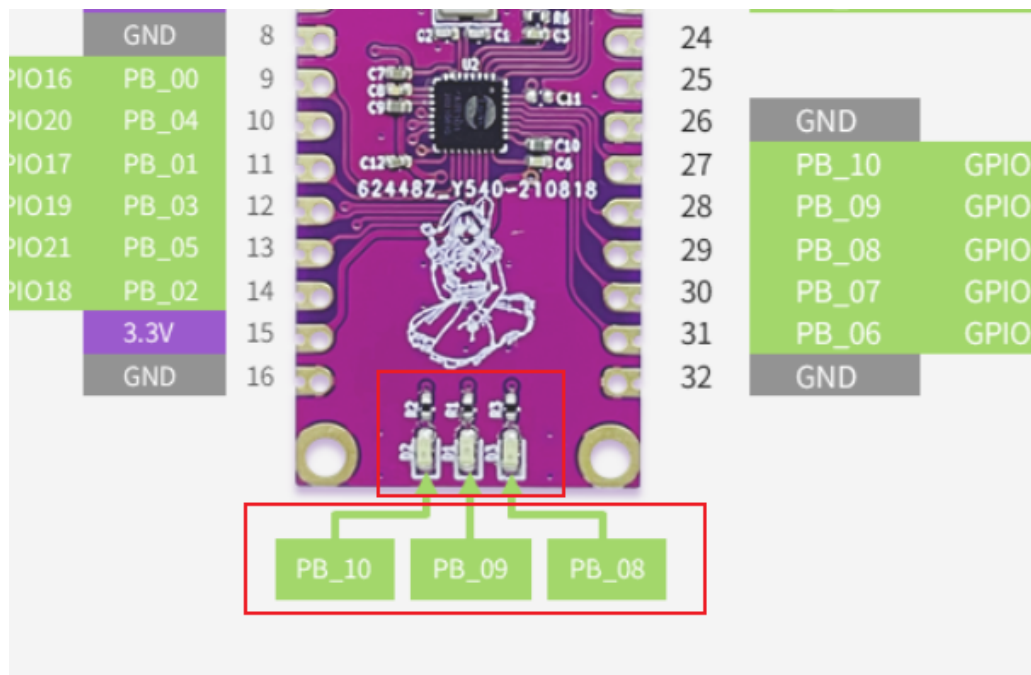
本章将使用GPIO接口控制开发板的LED灯进行闪烁，做个灯神

硬件准备

开发板一块，usb线一根

软件使用

根据pinout图可以找到开发板所带的三个灯分别为PB8，PB8，PB10控制，高电平亮起



在wiki中找一下gpio的接口[gpio - GPIO操作 - LuatOS 文档](#)

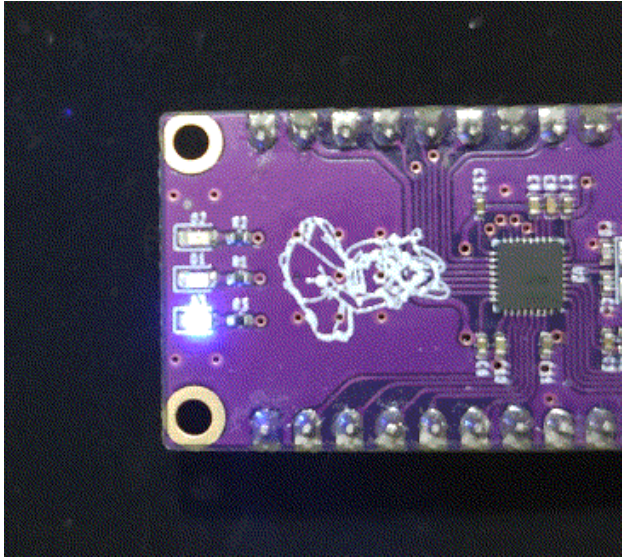
根据接口我们要先初始化，然后设置电平即可。

代码如下

```
local LED1 = gpio.setup(pin.PB08, 0) -- PB08输出模式
sys.taskInit(function()
```

```
while 1 do
  log.info("LED开启")
  LED1(0)
  sys.wait(1000)
  log.info("LED关闭")
  LED1(1)
  sys.wait(1000)
end
end)
```

烧录到设备观察现象



实践

编写代码让板子上带的三个灯依次亮起再依次熄灭。

GPIO输入-按键

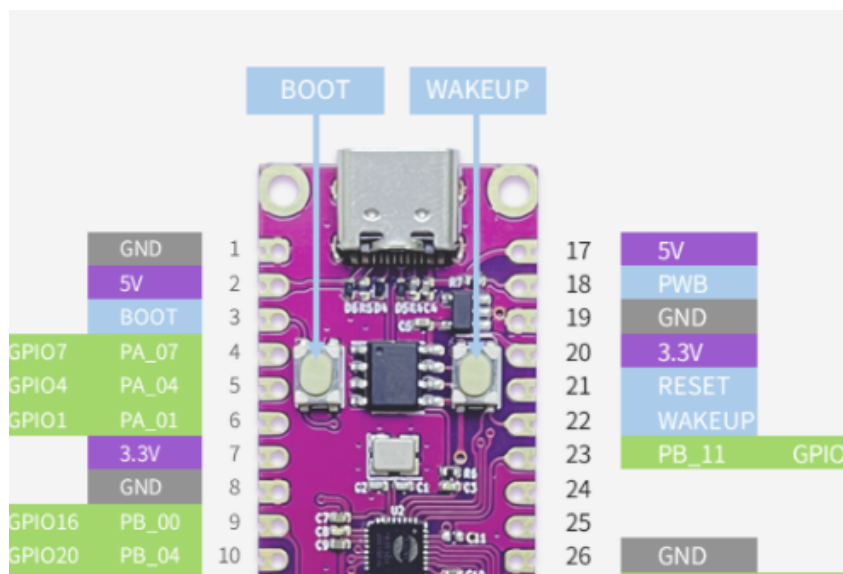
输出控制上一节学会了，趁热打铁学一下输入

硬件准备

开发板一块，usb线一根

软件使用

一样的套路，先看pinout图，按键2接了BOOT，也就是PA0，那就他了。



根据[gpio - GPIO操作](#) 的接口，简单写几行代码

```
gpio.setup(pin.PA00, function(val)
    log.info("PA0", val)
end, gpio.PULLUP)--按键按下接地，因此需要上拉
```

烧录，测试一下。按下boot键再松开，日志可以看的正常触发中断。

```
2021-12-25 18:53:22.352 #download 99%
2021-12-25 18:53:22.353
2021-12-25 18:53:22.552 #download 99%
2021-12-25 18:53:22.552
2021-12-25 18:53:22.831 download 100%
2021-12-25 18:53:22.831
2021-12-25 18:53:23.419 D/dbg [luat,log]2021-12-25 18:53:34.792 I/user.PA0 0
2021-12-25 18:53:34.973 I/user.PA0 1
2021-12-25 18:53:36.119 I/user.PA0 0
2021-12-25 18:53:37.097 I/user.PA0 1
2021-12-25 18:53:37.971 I/user.PA0 0
2021-12-25 18:53:38.232 I/user.PA0 0
2021-12-25 18:53:38.233 I/user.PA0 1
2021-12-25 18:53:38.994 I/user.PA0 0
2021-12-25 18:53:39.414 I/user.PA0 1
```

实践

编写代码，按键控制PB8的灯的状态，每次按下按键改变灯的状态。

PWM

本章将使用PWM驱动板载的led实现呼吸灯效果

简介

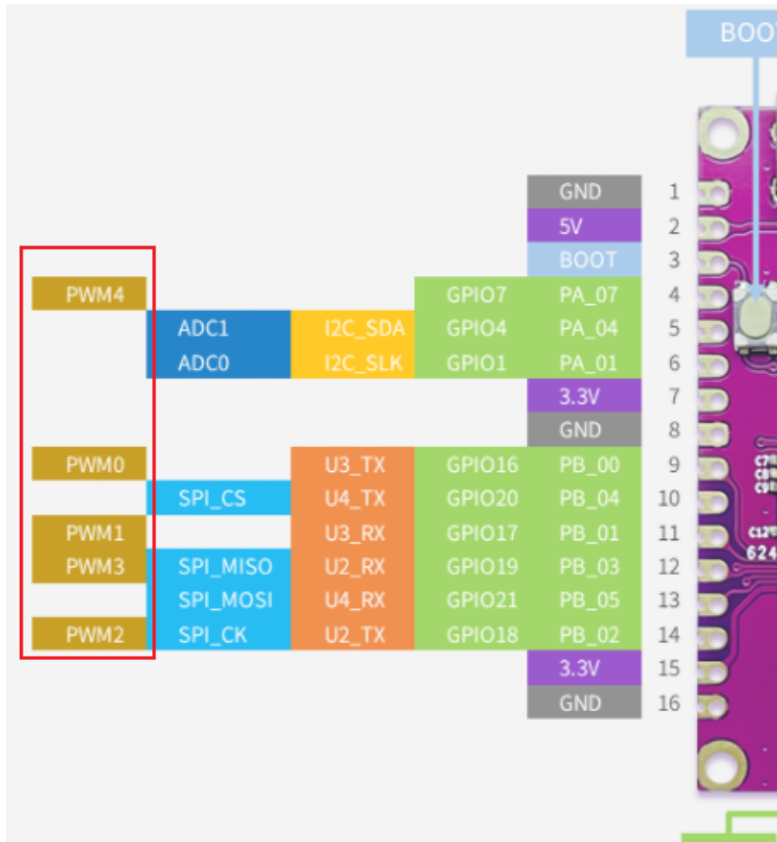
PWM(Pulse Width Modulation , 脉冲宽度调制) 是一种对模拟信号电平进行数字编码的方法，通过不同频率的脉冲使用方波的占空比用来对一个具体模拟信号的电平进行编码，使输出端得到一系列幅值相等的脉冲，用这些脉冲来代替所需要波形的设备。

硬件准备

开发板一块，usb线一根

软件使用

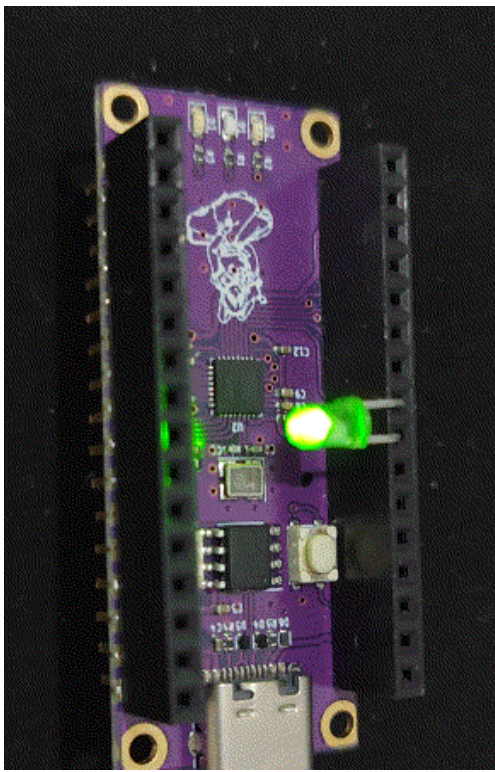
老规矩先看pinout，可以看到Air101有5路pwm引脚，我们就选PWM0直接接一个小灯到旁边的GND做个呼吸灯吧



查一下API发现PWM的使用很简单，就直接open就行，然后调节占空比就可以调节亮度，上代码。

```
sys.taskInit(function()
    while 1 do
        -- 仿呼吸灯效果
        log.info("pwm", ">>>>>")
        for i = 100,1,-1 do
            pwm.open(0, 1000, 100-i) -- 频率1000hz, 占空比0-100
            sys.wait(20)
        end
        sys.wait(1000)
        for i = 100,1,-1 do
            pwm.open(0, 1000, i)
            sys.wait(20)
        end
        gpio.setup(pin.PB00,0)
        sys.wait(1000)
    end
end)
```

烧录看效果，灯渐渐亮起又熄灭，好像在呼吸一样。



实践

编写代码实现三种呼吸效果，按下按键切换呼吸效果。

UART

本章讲学习如何配置串口，并通过串口发送收到的数据进行回环

简介

UART (Universal Asynchronous Receiver/Transmitter) 通用异步收发传输器，UART 作为异步串口通信协议的一种，工作原理是将传输数据的每个字符一位接一位地传输。是在应用程序开发过程中使用频率最高的数据总线。

UART 串口的特点是将数据一位一位地顺序传送，只要 2 根传输线就可以实现双向通信，一根线发送数据的同时用另一根线接收数据。UART 串口通信有几个重要的参数，分别是波特率、起始位、数据位、停止位和奇偶检验位，对于两个使用 UART 串口通信的端口，这些参数必须匹配，否则通信将无法完成。

硬件准备

开发板一块，usb转串口线一根

软件使用

查看文档[LuatOS 文档](#)可以看到，Air101有4个uart，其中uart0做下载调试用，所以我们选uart1吧。也就是PB6->TX然后PB7->RX把TX接串口线RX，RX接串口线TX，然后别忘了共地。

接好线以后开始写代码，根据[UART](#) 的API接口说明，首先我们初始化串口

```

local result = uart.setup(
    UART_ID, --串口id
    115200, --波特率
    8, --数据位
    1 --停止位
)

```

要做到串口数据回环首先要学习如何接收串口数据，接口里面的uart.on(id, event, func)注册串口事件回调，就是中断回调方式接收，收到以后再通过uart.write接口发出来就好了，理论分析完成，上代码

```

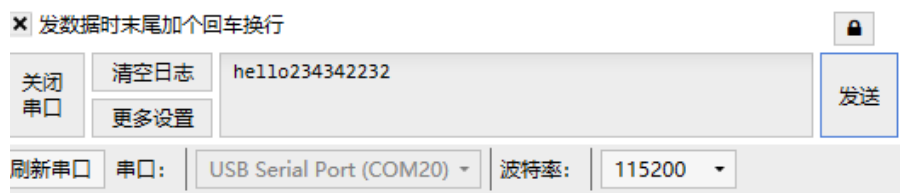
-- 串口ID,串口读缓冲区
local UART_ID, sendQueue = 1, {}
-- 串口超时, 串口准备好后发布的消息
--例子是100ms, 按需求改
local uarttimeout, recvReady = 100, "UART_RECV_ID"
--初始化
local result = uart.setup(
    UART_ID, --串口id
    115200, --波特率
    8, --数据位
    1 --停止位
)
uart.on(UART_ID, "receive", function(uid, length)
    local s
    while true do --保证读完不能丢包
        s = uart.read(uid, length)
        if #s == 0 then break end
        table.insert(sendQueue, s)
    end
    sys.timerStart(sys.publish, uarttimeout, recvReady)
end)
-- 向串口发送收到的字符串
sys.subscribe(recvReady, function()
    --拼接所有收到的数据
    local str = table.concat(sendQueue)
    -- 串口的数据读完后清空缓冲区
    sendQueue = {}
    --注意打印会影响运行速度, 调试完注释掉
    --log.info("uartTask.read length", #str, str:sub(1,100))
    uart.write(UART_ID, str) --回复
    --在这里处理接收到的数据, 这是例子
end)

```

对于不定长的数据接收一般采用超时断帧机制，也就是在收到前一个字节开启定时器，在一定时间内没有收到下一个字节就认为这是一包数据。

下载查看效果，收到不定长数据后回环输出。


```
[2021/11/04 14:05:51.2294] ← hello
[2021/11/04 14:05:51.4534] → hello
[2021/11/04 14:05:54.2993] ← hello23434
[2021/11/04 14:05:54.5290] → hello23434
[2021/11/04 14:05:56.2187] ← hello234342232
[2021/11/04 14:05:56.4597] → hello234342232
```



实践

编写代码，使用串口1收发指令，收到A,on打开LED1，收到A,off关闭LED1

ADC

本章将会向大家介绍LuatOS的ADC功能。将会实现使用Air101开发板读取内部温度并在日志中打印。

简介

模拟数字转换器即A/D转换器，或简称ADC，通常是指一个将模拟信号转变为数字信号电子元件。通常的模数转换器是将一个输入电压信号转换为一个输出的数字信号。

Air101芯片具有两路16位ADC，最高采样率1KHz。

硬件准备

开发板一块，可调电源一个

软件使用

这次我们先找文档[LuatOS 文档](#)可以看到除了可以读取外接的IO电压之外，还可以读取内部的温度和供电电压。我们这次选取ADC0，也就是PA1引脚。

我们给PA1脚接可调电源（一定注意电压不要超过2.4V）

根据[ADC库](#)的接口，简单写几行代码

```
sys.taskInit(function()
```

```

while 1 do
  adc.open(0) -- 模块上的ADC0脚-PA1, 0~2.4v,不要超过范围使用!!!
  adc.open(10) -- CPU温度
  adc.open(11) -- VBAT电压,最新代码才支持
  sys.wait(500)
  log.debug("adc", "adc0", adc.read(0))
  log.debug("adc", "adc_temp", adc.read(10))
  log.debug("adc", "vbat", adc.read(11))
  sys.wait(500)
end
end)

```

烧录，观察日志

2021-12-25 19:24:29.073	D/user.adc	adc0	49140.00	93.00000
2021-12-25 19:24:29.079	D/user.adc	adc_temp	41648.00	41648.00
2021-12-25 19:24:29.103	D/user.adc	vbat	3218.000	3218.000
2021-12-25 19:24:30.109	D/user.adc	adc0	46460.00	10.00000
2021-12-25 19:24:30.115	D/user.adc	adc_temp	41696.00	41696.00
2021-12-25 19:24:30.138	D/user.adc	vbat	3218.000	3218.000
2021-12-25 19:24:31.145	D/user.adc	adc0	46036.00	0.00000e-308
2021-12-25 19:24:31.151	D/user.adc	adc_temp	41423.00	41423.00
2021-12-25 19:24:31.174	D/user.adc	vbat	3218.000	3218.000
2021-12-25 19:24:32.181	D/user.adc	adc0	46380.00	7.000000
2021-12-25 19:24:32.187	D/user.adc	adc_temp	41358.00	41358.00
2021-12-25 19:24:32.211	D/user.adc	vbat	3218.000	3218.000
2021-12-25 19:24:33.216	D/user.adc	adc0	47772.00	50.00000
2021-12-25 19:24:33.222	D/user.adc	adc_temp	41262.00	41262.00
2021-12-25 19:24:33.246	D/user.adc	vbat	3218.000	3218.000

实测ADC有一定误差，建议之作阈值检测之类的用。

实践

编写代码实现当芯片内部温度大于40度时亮起PB10这个LED

I2C

本章将会向大家介绍LuatOS的I2C功能。将会实现使用Air101开发板I2C读取SHT30传感器的温湿度

简介

I2C（Inter Integrated Circuit）总线是 PHILIPS 公司开发的一种半双工、双向二线制同步串行总线。I2C 总线传输数据时只需两根信号线，一根是双向数据线 SDA（serial data），另一根是双向时钟线 SCL（serial clock）

硬件准备

开发板一块，SHT30传感器一个

软件使用

首先还是查文档[LuatOS 文档](#)查阅文档可知PA1为SCL引脚，PA4为SDA引脚。我们将传感器按照这个定义连到板子上。

开始写代码，先找接口说明[i2c - I2C操作 - LuatOS 文档](#)，先初始化，需要地址，查阅SHT30手册，地址是由ADDR这个引脚电平决定，我这个模块悬空，就是0x44

Please note that the I2C address is represented through the 7 MSBs of the I2C read or write header. The LSB switches between read or write header. The wiring for the default address is shown in Table 8 and Figure 11. The ADDR pin must not be left floating. Please note that only the 7 MSBs of the I2C Read/Write header constitute the I2C Address.

SHT3x-DIS	I2C Address in Hex. representation	Condition
I2C address A	0x44 (default)	ADDR (pin 2) connected to logic low
I2C address B	0x45	ADDR (pin 2) connected to logic high

知道地址了，接下来看一下怎么测量温度，手册说连续测量需要发命令0x2C06，参照LuatOS的接口，直接i2c.send()就完事

Condition		Hex. code	
Repeatability	Clock stretching	MSB	LSB
High	enabled	0x2C	06
Medium			0D
Low			10
High	disabled	0x24	00
Medium			0B
Low			16

e.g. 0x2C06: high repeatability measurement with clock stretching enabled

然后接收数据，根据手册的公式计算实际温湿度。

Relative humidity conversion formula (result in %RH):

$$RH = 100 \cdot \frac{S_{RH}}{2^{16} - 1}$$

Temperature conversion formula (result in °C & °F):

$$T [^{\circ}C] = -45 + 175 \cdot \frac{S_T}{2^{16} - 1}$$

$$T [^{\circ}F] = -49 + 315 \cdot \frac{S_T}{2^{16} - 1}$$

S_{RH} and S_T denote the raw sensor output for humidity and temperature, respectively. The formulas work only correctly when S_{RH} and S_T are used in decimal representation.

代码如下

```
sys.taskInit(function()  
  -- 初始化i2c, 使用id为0  
  if i2c.setup(0, i2c.FAST, 0x44) == 1 then  
    log.info("存在 i2c0")  
  else  
    i2c.close(0) -- 关掉  
  end  
  while 1 do
```

```

local w = i2c.send(0, 0x44, string.char(0x2c, 0x06)) -- 发送单次采集命令
sys.wait(10) -- 等待采集
local r = i2c.recv(0, 0x44, 6) -- 读取数据采集结果
log.info("recv", r:toHex())
local a, b, c, d, e, f, g = string.unpack("BBBBBB", r)
log.info("a", a, b, c, d, e, f, g)
local t = ((4375 * (a * 256 + b)) / 16384) - 4500 -- 根据SHT30传感器手册给的公式
local h = ((2500 * (d * 256 + e)) / 16384)
log.warn("这里是温度", t / 100) -- 打印温度
log.warn("这里是湿度", h / 100) -- 打印湿度
sys.wait(1000)

end
end)

```

烧录查看结果，屋里开着空调挺暖和的

```

2021-12-25 19:31:03.404 I/user.recv 675ADF4F7B83 12
2021-12-25 19:31:03.405 I/user.a 103 90 223 79 123 131 7
2021-12-25 19:31:03.406 W/user.这里是温度 25.65048
2021-12-25 19:31:03.408 W/user.这里是湿度 31.04706
2021-12-25 19:31:04.421 I/user.recv 675F2A4F69A2 12
2021-12-25 19:31:04.422 I/user.a 103 95 42 79 105 162 7
2021-12-25 19:31:04.423 W/user.这里是温度 25.66383
2021-12-25 19:31:04.425 W/user.这里是湿度 31.01959
2021-12-25 19:31:05.438 I/user.recv 6764054F1ECD 12
2021-12-25 19:31:05.439 I/user.a 103 100 5 79 30 205 7
2021-12-25 19:31:05.440 W/user.这里是温度 25.67719
2021-12-25 19:31:05.442 W/user.这里是湿度 30.90515

```

实践

编写代码尝试驱动其他I2C器件。

SPI

本章将使用Air101的硬件SPI读取flash的ID并打印。

简介

SPI是串行外设接口（Serial Peripheral Interface）的缩写，是一种高速的，全双工，同步的通信总线，设备分为主机和从机，目前Air101的SPI仅能作为主机使用。

硬件准备

开发板一块，SPI的flash一个。

软件使用

先找引脚图研究怎么接线，查文档[LuatOS 文档](#)查阅文档可知接线方式，也就是PB2接CLK，PB5接MOSI，PB3接MISO，PB4接CS。接下来研究如何驱动。

我用的flash是W25Q128，根据手册可以查到有一个读flash的型号的通用命令0x9F，正常的话W25Q128应该会返回0xEF4018

7.2.1 Manufacturer and Device Identification

MANUFACTURER ID	(MF7-MF0)	
Winbond Serial Flash	EFh	
Device ID	(ID7-ID0)	(ID15-ID0)
Instruction	ABh, 90h, 92h, 94h	9Fh
W25Q128BV	17h	4018h

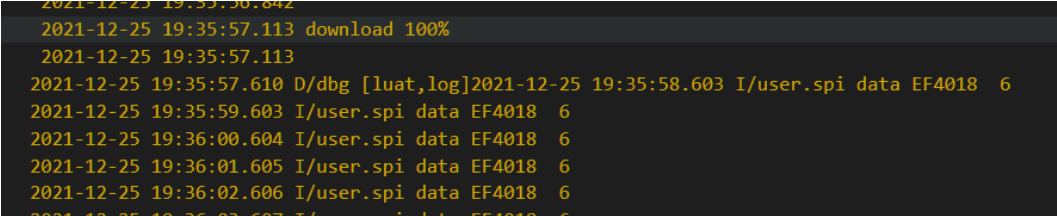
我们参照[API手册](#)先初始化SPI，创建一个SPI对象spi_device = spi.deviceSetup(0,20,0,0,8,2000000,spi.MSB,1,1)

发0x9F指令，读取返回的3个字节recv = spi_device:transfer(string.char(0x9f),nil,3)

整体代码如下

```
sys.taskInit(  
  function()  
    local spi_device = spi.deviceSetup(0,pin.PB04,0,0,8,2000000,spi.MSB,1,1)  
    while 1 do  
      local recv = spi_device:transfer(string.char(0x9f),nil,3)  
      log.info("spi data",recv:toHex())  
      sys.wait(1000)  
    end  
  end  
)
```

下载到设备，可以看到日志中打印的值是EF4018符合手册说明，SPI通信正常



实践

编写代码尝试驱动其他SPI器件。

SFUD-外置flash

本章将使用Air101的硬件SPI挂载flash到文件系统，直接读写

简介

上一章我们学会了用SPI读取外置flash的ID，实际通常外挂的spi flash的读写指令都是兼容的，我们在日常使用的过程中如果直接使用spi通过指令对flash进行读写会很麻烦，于是乎LuatOS就设计了一套接口，将外置的spi flash通过这套接口实现抽象读写，并对接Lua的io实现简单读写。

硬件准备

开发板一块，SPI的flash一个

软件使用

接线图看SPI那一章即可不在赘述

参考[sfud 文档](#)写个简单的demo,大致思路就是先给sfud创建一个spi的硬件设备对象，然后通过sfud的抽象接口操作spi实现对flash的操作

```
local spi_flash = spi.deviceSetup(0,20,0,0,8,2000000,spi.MSB,1,1)--PB6
log.info("sfud.init",sfud.init(spi_flash))
log.info("sfud.getDeviceNum",sfud.getDeviceNum())
local sfud_device = sfud.getDeviceTable()
log.info("sfud.write",sfud.write(sfud_device,1024,"sfud"))
log.info("sfud.read",sfud.read(sfud_device,1024,4))
```

先测试一下能不能正常连接设备，烧录。查看日志可以看到sfud的库在初始化的时候自动帮我们获取了flash的厂商、大小等信息，我们通过sfud.write()和read()接口读写也都正常。

```
2021-12-25 19:59:04.733 D/luat.sfud write granularity is 64 bytes or larger.
2021-12-25 19:59:04.740 D/luat.sfut flash status register is non-v.
2021-12-25 19:59:04.741 D/luat.sfud 3-Byte only addressing.
2021-12-25 19:59:04.741 D/luat.sfud Capacity is 16777216 Bytes.
2021-12-25 19:59:04.742 D/luat.sfud Flash device supports 4KB block erase. Command is 0x20.
2021-12-25 19:59:04.743 D/luat.sfud Flash device supports 32KB block erase. Command is 0x52.
2021-12-25 19:59:04.744 D/luat.sfud Flash device supports 64KB block erase. Command is 0xD8.
2021-12-25 19:59:04.745 D/luat.sfud Find a Winbond flash chip. Size is 16777216 bytes.
2021-12-25 19:59:04.745 D/luat.sfud Flash device reset success.
2021-12-25 19:59:04.746 D/luat.sfud flash device is initialize success.
2021-12-25 19:59:04.746 I/user.sfud.init      true
2021-12-25 19:59:04.830 I/user.sfud.getDeviceNum      1
2021-12-25 19:59:04.981 I/user.sfud.eraseWrite    0
2021-12-25 19:59:05.083 I/user.sfud.read      sfud
```

可是这样我们就满足了吗？不！我们要把flash变成文件系统的一部分直接通过lua原生接口读写。根据API的说明我们可以找到sfud.mount()这个函数，通过他就可以直接挂载到文件系统了，mount很像linux的操作，有没有。

接下来实践一下，在前面的代码基础上增加如下代码：

```
log.info("sfud.mount",sfud.mount(sfud_device,"/sfud"))
log.info("fsstat", fs.fsstat("/sfud"))
local f = io.open ("/sfud/a.txt", "wb")
f:write(string.rep("1234567890", 100))
f:close()
log.info("fsize", fs.fsize("/sfud/a.txt"))
f = io.open ("/sfud/a.txt", "rb")
local data = f:read("*a")
log.info("fs", "data", data)
```

我们挂载到文件系统后，新建一个文件写入1000字节数据，然后读一下文件大小，再把文件内容读出来。

烧录看一下效果

[illegible]

完全符合预期，可以直接通过文件系统操作flash，是不是很方便。

实践

编写代码测试flash读写速度

SDIO

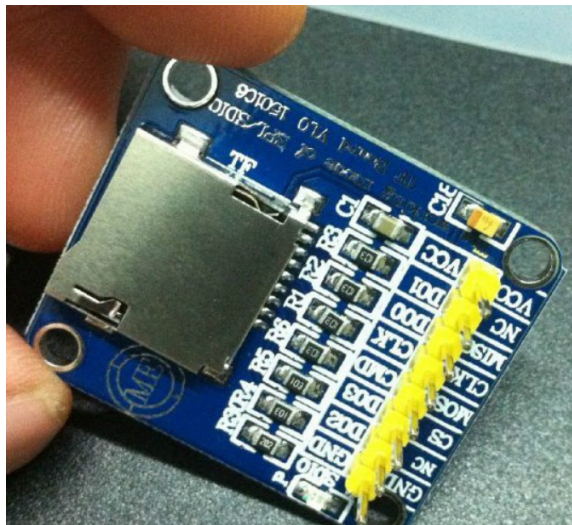
本章将会学习使用SDIO接口驱动TF卡，并把TF卡挂载到文件系统进行读写

简介

上一章我们学会了用sfud这个接口直接挂载flash，可是spi就俩数据线有点慢，而且flash容量有限。这时候就想到更高速率的SDIO接口了，SDIO接口设计之初就是为了读写SD卡，后来被拓展很多高速通信中也借用这个接口。

硬件准备

开发板一块，支持SDIO的tf卡或SD卡模块。一定要类似下图这种支持SDIO的4数据线的。



不要这种只支持SPI的



软件使用

先看查文档[LuatOS 文档](#)研究一下怎么接线

引脚	功能
PB_06	GPIO_22 / UART1_TX / SDIO_CLK
PB_07	GPIO_23 / UART1_RX / SDIO_CMD
PB_08	GPIO_24 / SDIO_D0
PB_09	GPIO_25 / SDIO_D1
PB_10	GPIO_26 / SDIO_D2
PB_11	GPIO_27 / SDIO_D3

按照这个方式接好模块，别忘了放卡。

接下找[API 文档](#)，和sfud一个套路，初始化硬件，读写测试（最好别这么测，直接读写sd卡会导致分区表丢失，插电上可能不识别），我就不测了

直接上挂载的部分，除了前两句，剩下的全是原生接口，和sfud一样，挂载完直接使用，完整代码在demo的sdio文件夹

```
sys.taskInit(  
    function()  
        sys.wait(2000)  
        --延时一下，防止日志看不到  
        sdio.init(0)  
        sdio.sd_mount(0, "/sd")  
        local f = io.open("/sd/a.txt", "wb")  
        f:write(string.rep("1234567890", 100))  
        f:close()  
        log.info("fsize", fs.fsize("/sd/a.txt"))  
        f = io.open("/sd/a.txt", "rb")  
        local data = f:read("*a")  
        log.info("fs", "data", data)  
    end  
)
```

烧录看看结果

[illegible]

SD卡使用第一次这么简单，两行代码就搞定。

实践

编写代码记录串口收到的数据到SD卡中

LCD

这一章我们将学习如何驱动LCD屏幕，并在屏幕上显示汉字和图形

简介

通常驱动屏幕是一件很复杂的事情，因为LCD屏幕有很多指令，显示数据还要制作字库。LuatOS将LCD进行了封装，内置了多种常用的屏幕驱动，不在列表的也可以通过Lua脚本配置指令进行驱动。使用虚拟显存的思想，将硬件驱动和图像绘制抽离，使用更简单方便。

硬件准备

开发板一块，SPI的屏幕一个，我用的gc9306（240*320）

软件使用

老规矩先看查文档[LuatOS 文档](#)研究一下怎么接线，只需前两个引脚必须用这个表的解法，后面的可以随便接，我推荐按我的接

功能	引脚
SCL	PB2
SDA	PB5
RES	PB3
DC	PB1
CS	PB4
BL	PB0

接下来老规矩看API思路都一样，先初始化硬件，在搞其他。按照接口的说明，我们要先初始话一个spi对象，然后根据硬件配置RES和DC等引脚，引脚列表前面我写了，所以初始化代码如下

```
local spi_lcd = spi.deviceSetup(0, pin.PB04, 0, 0, 8, 20 * 1000 * 1000, spi.MSB, 1, 1)
log.info(
    "lcd.init",
    lcd.init(
        "gc9306",
        {
            port = "device",
            pin_dc = pin.PB01,
            pin_pwr = pin.PB00,
            pin_rst = pin.PB03,
            direction = 0,
            w = 240,
            h = 320,
            xoffset = 0,
            yoffset = 0
        },
        spi_lcd
    )
)
```

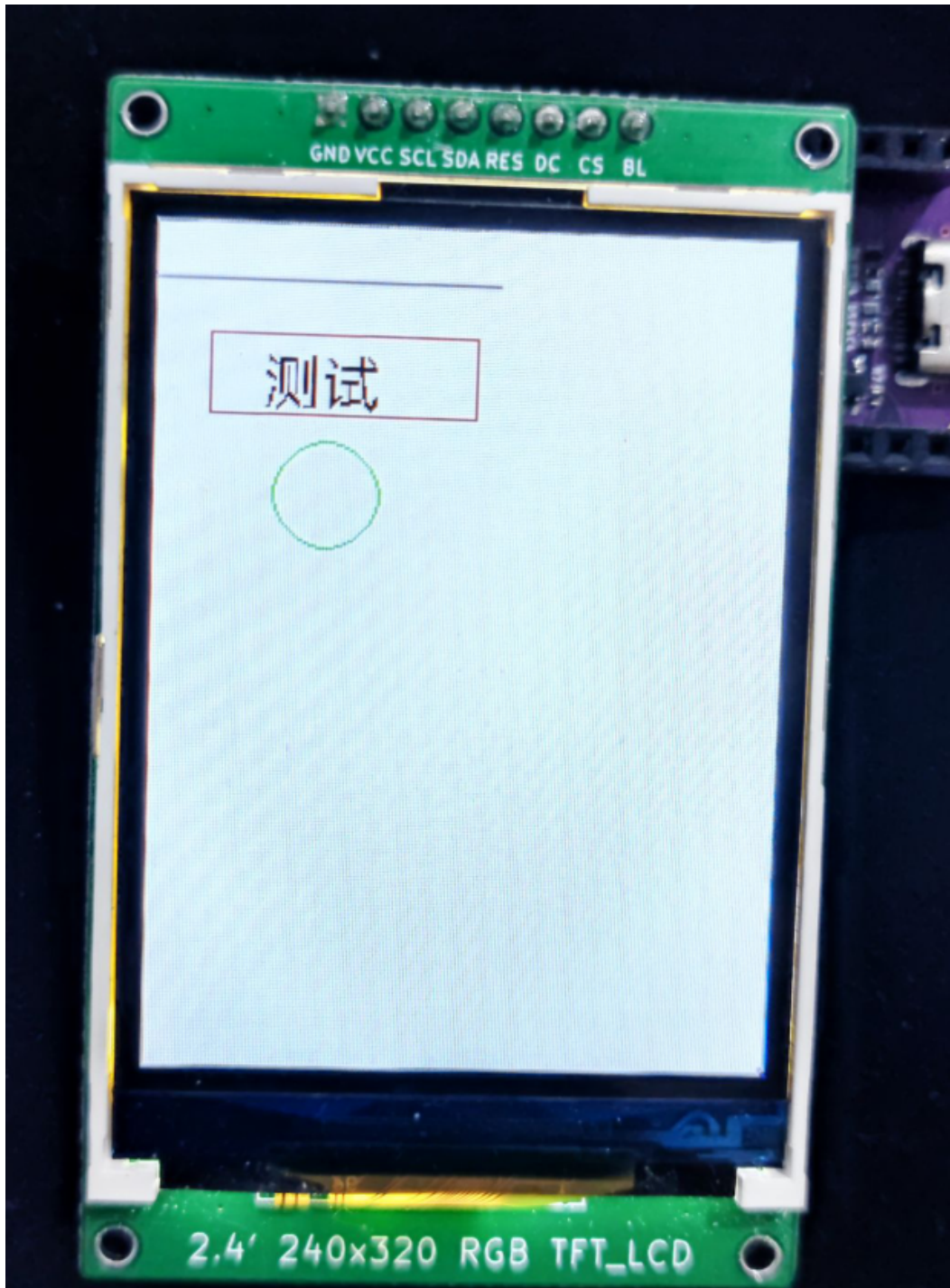


```
)  
)
```

接下来研究怎么显示一点东西，就简单画个框画个圆显示一下汉字吧，API里写的比较清楚了，这里就不多解释了。其中设置字体那个地方的字体是可以在前面讲过的本地编译或者云编译的时候选择，选的不一样这里设置也不一样，需要注意。

```
log.info("lcd.drawLine", lcd.drawLine(0,20,128,20,0x001F))  
log.info("lcd.drawRectangle", lcd.drawRectangle(20,40,120,70,0xF800))  
log.info("lcd.drawCircle", lcd.drawCircle(64,100,20,0x0CE0))  
lcd.setFont(lcd.font_opposansm16_chinese)  
lcd.drawStr(40,66,"测试")
```

烧录查看效果



换个屏试试，只需要改一下初始化参数的屏幕型号gc9306改成ili9341， ili9341也就驱动起来了

看看效果



LuatOS对显示设备的抽象很值得学习

实践

编写代码把本章接线顺序的表格显示在屏幕上

LVGL

本章将简单介绍如何使用LVGL显示更美观的画面

简介

上一章学了LCD的库，可以顺利的画点画线显示汉字了，可是想把界面做好看，这些简单的接口还是不能满足需求。于是乎就出现了很多图形库，LuatoS集成了LVGL图形库，可以利用大量控件写出美观的界面。

硬件准备

开发板一块，SPI的屏幕一个，我用的gc9306（240*320）

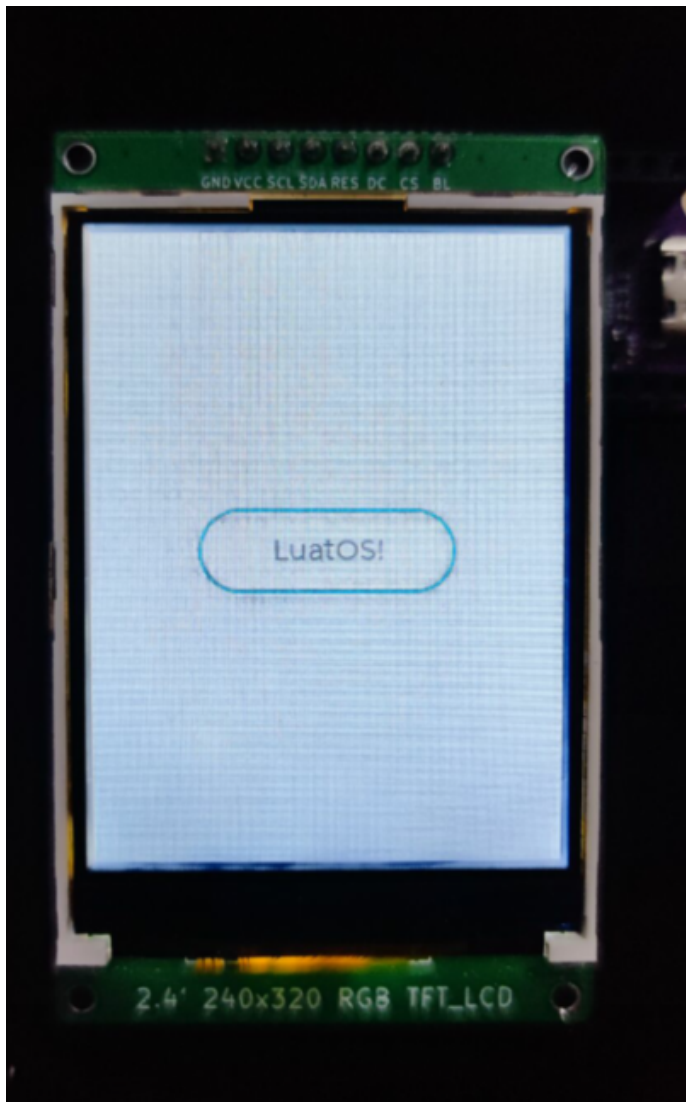
软件使用

注意要使用固件带LVGL后缀的固件，或者自行云编译。

接线看LCD那一章不重复，LVGL作为图形库首先还是LCD那套初始化屏幕，然后就是使用LVGL渲染屏幕内容。直接上代码了

```
log.info("lvgl", lvgl.init())
lvgl.disp_set_bg_color(nil, 0xFFFFFF)
local scr = lvgl.obj_create(nil, nil)
local btn = lvgl.btn_create(scr)
lvgl.obj_align(btn, lvgl.scr_act(), lvgl.ALIGN_CENTER, 0, 0)
local label = lvgl.label_create(btn)
lvgl.label_set_text(label, "LuatoS!")
lvgl.scr_load(scr)
```

看一下效果，这按键就用画线做出来的好看多了



更多的LVGL使用教程可以参考API和lvgl官方文档

实践

编写代码使用LVGL实现LCD那一章的实践要求